

Public comments: paragraph corpus and structural topic model

```
# The only block you edit. Every analyst decision is a line here.
tx <- list(
  seed          = 2026L,
  proj          = "text_mining", # project folder under the here() root;
                                # the one string to change when reusing
                                # this workflow on another corpus

  para_split    = "\\r?\\n+", # blank-line separated? "\\r?\\n{2,}"
  para_min_tokens = 15L,      # body fragments below this merge BACKWARD
  heading_max_tokens = 12L,   # headings merge FORWARD (see atomization)
  min_docfreq    = 5L,        # DTM vocabulary floor
  run_searchK    = TRUE,      # TRUE for the long K-selection run
  K_grid         = seq(10, 40, 3),
  K              = 25L,        # set from the searchK diagnostics
  spline_df      = 5L,
  facets_per_page = 9L,        # topic trend panels per page
  workers        = 6L
)

suppressPackageStartupMessages({
  library(here); library(stm); library(quanteda); library(tidytext)
  library(splines)
  library(furrr); library(viridis)
  library(tidyverse) # tidyverse LAST so its verbs win
})
theme_set(theme_minimal(base_size = 12))
# Tables must not silently lose rows in the rendered PDF: knitr truncates long
# tables by default, and a 25-topic summary is past that limit.
options(knitr.table.max.print = Inf, knitr.kable.max_rows = Inf,
        max.print = 1e6)
set.seed(tx$seed)
# All paths are anchored with here(), so chunks run the same whether you
# render from the project root or run them interactively from code/.
# Layout: <here()>/text_mining/{code, input, outputs}.
p_out <- function(f) here::here(tx$proj, "outputs", f)
dir.create(here::here(tx$proj, "outputs"), showWarnings = FALSE,
           recursive = TRUE)

# Checkpoints: compute once, reload after. Delete the file to re-fit.
cache_rds <- function(path, expr) {
  if (file.exists(path)) return(readRDS(path))
  x <- force(expr); saveRDS(x, path); x
}
```

```

}
cache_csv <- function(path, expr) {
  if (file.exists(path)) return(readr::read_csv(path, show_col_types = FALSE))
  x <- force(expr); readr::write_csv(x, path); x
}

```

SIMULATED CORPUS (delete this section at work)

Plants 25 topics with distinct vocabularies and per-topic year trajectories, so the pipeline can be scored on recovery of structure it was not shown. Script 2 reuses the planted labels to validate its estimators.

```

sim <- local({
  K <- 25L; years <- 2000:2025
  vocab <- purrr::map(seq_len(K), function(k)
    str_c("t", k, "term", seq_len(12)))
  filler <- str_c("common", seq_len(40))
  # topic prevalence drifts linearly in year, each topic with its own slope
  slope <- seq(-1, 1, length.out = K)
  make_comment <- function(i) {
    yr <- sample(years, 1)
    pr <- exp(slope * (yr - 2012) / 12); pr <- pr / sum(pr)
    n_p <- sample(4:12, 1)
    ks <- sample.int(K, n_p, replace = TRUE, prob = pr)
    body <- purrr::map_chr(ks, function(k) str_c(
      str_c(sample(vocab[[k]], 14, replace = TRUE), collapse = " "), " ",
      str_c(sample(filler, 10, replace = TRUE), collapse = " ")))
    tibble(atom_id = sprintf("A%05d", i), Year = yr,
      body_english = str_c(body, collapse = "\n"),
      true_topics = str_c(ks, collapse = ","))
  }
  purrr::map(seq_len(1040), make_comment) |> purrr::list_rbind()
})
corpus_raw <- sim |> dplyr::select(atom_id, Year, body_english)
readr::write_csv(sim |> dplyr::select(atom_id, true_topics),
  p_out("sim_truth_comments.csv"))

```

Corpus validation (keep this at work)

Stops the render with a named error on a bad read, which is cheaper than debugging a silent failure five chunks later.

```

miss <- setdiff(c("atom_id", "Year", "body_english"), names(corpus_raw))
if (length(miss)) stop("corpus_raw missing column(s): ",
  str_c(miss, collapse = ", "))
stopifnot(

```

```

"atom_id must be unique"      = !anyDuplicated(corpus_raw$atom_id),
"Year must be integer-valued" = all(corpus_raw$Year ==
                                   as.integer(corpus_raw$Year)),
"body_english must be character" = is.character(corpus_raw$body_english))
n0 <- nrow(corpus_raw)
corpus_raw <- corpus_raw |>
  dplyr::filter(!is.na(Year), !is.na(body_english),
               str_squish(body_english) != "")
glue::glue("Validation: kept {nrow(corpus_raw)} of {n0} comments, ",
           "{min(corpus_raw$Year)}-{max(corpus_raw$Year)}")

```

Validation: kept 1040 of 1040 comments, 2000-2025

Newline diagnostic

The atomizer can only split on what the source uses to mark paragraphs, and Excel-origin text (in-cell breaks survive as \n) carries three conventions needing three responses. Rules: mostly line breaks present, keep the default; blank lines dominant **and** the visible sample shows single breaks used as soft wraps, set `tx$para_split = "\\r?\n{2,}"`; almost no newlines, stop, since sentence chunking changes the measurement unit and is a design decision.

```

corpus_raw |>
  dplyr::mutate(b = str_replace_all(body_english, "\\r\\n", "\\n")) |>
  dplyr::summarise(comments = dplyr::n(),
                  zero_newlines = sum(str_count(b, "\\n") == 0),
                  has_blank_lines = sum(str_count(b, "\\n{2,}") > 0),
                  median_breaks = median(str_count(b, "\\n"))) |>
  knitr::kable(caption = "Line-break conventions; apply the decision rules above
before the long render.")

```

comments	zero_newlines	has_blank_lines	median_breaks
1040	0	0	7

Table 1: Line-break conventions; apply the decision rules above before the long render.

```

corpus_raw |> dplyr::slice_sample(n = 2) |> dplyr::pull(body_english) |>
  str_replace_all("\\r?\\n", "<NL>\\n") |>
  str_c(collapse = "\\n\\n==== NEXT ====\\n\\n") |> writeLines()

```

```

t23term2 t23term10 t23term3 t23term2 t23term8 t23term5 t23term7 t23term4 t23term8
t23term8 t23term5 t23term5 t23term5 t23term12 common7 common8 common35 common6
common6 common19 common16 common36 common3 common32<NL>
t5term5 t5term2 t5term5 t5term3 t5term1 t5term1 t5term10 t5term7 t5term3 t5term5
t5term3 t5term11 t5term9 t5term11 common3 common24 common7 common28 common16

```

```

common12 common35 common21 common39 common19<NL>
t11term5 t11term2 t11term4 t11term7 t11term7 t11term4 t11term5 t11term1 t11term10
t11term1 t11term9 t11term12 t11term12 t11term7 common10 common21 common17 common4
common15 common38 common16 common4 common7 common31<NL>
t14term7 t14term10 t14term4 t14term11 t14term5 t14term2 t14term6 t14term3
t14term1 t14term9 t14term12 t14term7 t14term12 t14term4 common5 common24 common37
common3 common6 common28 common14 common5 common5 common9<NL>
t24term3 t24term7 t24term5 t24term8 t24term9 t24term9 t24term8 t24term7 t24term8
t24term8 t24term6 t24term3 t24term6 t24term10 common30 common6 common21 common5
common31 common10 common39 common22 common6 common9<NL>
t11term7 t11term8 t11term12 t11term7 t11term5 t11term11 t11term2 t11term4
t11term3 t11term12 t11term3 t11term12 t11term9 t11term7 common13 common2 common13
common30 common38 common34 common17 common34 common34 common22<NL>
t19term12 t19term6 t19term6 t19term1 t19term12 t19term8 t19term8 t19term7
t19term11 t19term4 t19term3 t19term7 t19term4 t19term2 common10 common6 common21
common39 common5 common40 common9 common38 common7 common8

==== NEXT ====

t25term9 t25term7 t25term10 t25term10 t25term6 t25term7 t25term8 t25term4
t25term11 t25term11 t25term5 t25term5 t25term9 t25term8 common16 common27
common13 common19 common1 common24 common24 common27 common15 common8<NL>
t1term12 t1term10 t1term1 t1term10 t1term6 t1term4 t1term2 t1term10 t1term11
t1term4 t1term9 t1term8 t1term12 t1term1 common22 common7 common12 common36
common33 common14 common10 common18 common9 common39<NL>
t14term4 t14term8 t14term2 t14term12 t14term4 t14term11 t14term2 t14term5
t14term10 t14term10 t14term9 t14term9 t14term1 t14term6 common28 common23
common11 common36 common23 common36 common20 common40 common8 common20<NL>
t5term1 t5term5 t5term8 t5term2 t5term10 t5term12 t5term11 t5term10 t5term9
t5term5 t5term2 t5term3 t5term12 t5term12 common8 common14 common14 common17
common31 common16 common7 common30 common4 common4<NL>
t17term12 t17term1 t17term7 t17term3 t17term9 t17term4 t17term5 t17term10
t17term9 t17term12 t17term3 t17term12 t17term10 t17term4 common8 common12
common28 common6 common26 common9 common27 common35 common12 common34<NL>
t4term6 t4term11 t4term5 t4term10 t4term7 t4term5 t4term7 t4term9 t4term11
t4term2 t4term9 t4term2 t4term12 t4term10 common26 common40 common15 common31
common23 common24 common29 common24 common36 common18

```

Paragraph atomization

The unit of analysis is the paragraph: a comment spans several topics, and a topic model’s “document” is whatever unit the DTM rows are. Merging is **direction-aware**, because short blocks come in two kinds that belong in opposite directions. Headings (at most 12 tokens, no sentence-ending punctuation) merge **forward** into the paragraph they introduce, since a naive rule would glue a subtitle’s words onto the *preceding* paragraph and misfile its topic signal; a trailing heading falls back to the last body paragraph. Short body fragments that do end like sentences merge **backward** into what they continue, absorbing codas, salutations, and signature lines.

```

count_tokens <- function(x) str_count(x, "\\S+")

paragraphs <- cache_csv(p_out("paragraphs.csv"), {
  # PASS 1 (forward): rev(cumsum(rev(!heading))) gives each body block an id
  # that decreases down the document and gives a heading the id of the NEXT
  # body block; trailing headings get 0, folded in by pmax(1L).
  corpus_raw |>
    dplyr::mutate(para_raw = str_split(body_english, tx$para_split)) |>
    dplyr::select(atom_id, Year, para_raw) |>
    tidyr::unnest(para_raw) |>
    dplyr::mutate(para_raw = str_squish(para_raw)) |>
    dplyr::filter(para_raw != "") |>
    dplyr::group_by(atom_id, Year) |>
    dplyr::mutate(
      heading = count_tokens(para_raw) <= tx$heading_max_tokens &
        !str_detect(para_raw, "[.!?]['\"]\\]\\*$"),
      fwd = rev(cumsum(rev(!heading))) |> pmax(1L)) |>
    dplyr::group_by(atom_id, Year, fwd) |>
    dplyr::summarise(para_raw = str_c(para_raw, collapse = " "),
      .groups = "drop") |>
    dplyr::arrange(atom_id, Year, dplyr::desc(fwd)) |>
  # PASS 2 (backward): short body fragments join the preceding paragraph.
  dplyr::group_by(atom_id, Year) |>
  dplyr::mutate(full = count_tokens(para_raw) >= tx$para_min_tokens,
    blk = cummax(cumsum(full) * full) |> pmax(1L)) |>
  dplyr::group_by(atom_id, Year, blk) |>
  dplyr::summarise(text = str_c(para_raw, collapse = " "),
    .groups = "drop_last") |>
  dplyr::mutate(para_id = dplyr::row_number(),
    para_uid = str_c(atom_id, "-", sprintf("%02d", para_id)),
    n_tokens = count_tokens(text)) |>
  dplyr::ungroup() |>
  dplyr::filter(n_tokens >= tx$para_min_tokens) |>
  dplyr::select(para_uid, atom_id, para_id, Year, n_tokens, text)
})
glue::glue("{nrow(paragraphs)} paragraphs from ",
  "{dplyr::n_distinct(paragraphs$atom_id)} comments")

```

8236 paragraphs from 1040 comments

Corpus description

```

paragraphs |>
  dplyr::count(Year) |>
  ggplot(aes(Year, n)) +

```

```
geom_col(fill = viridis(1, begin = 0.35)) +
labs(x = NULL, y = "paragraphs")
```

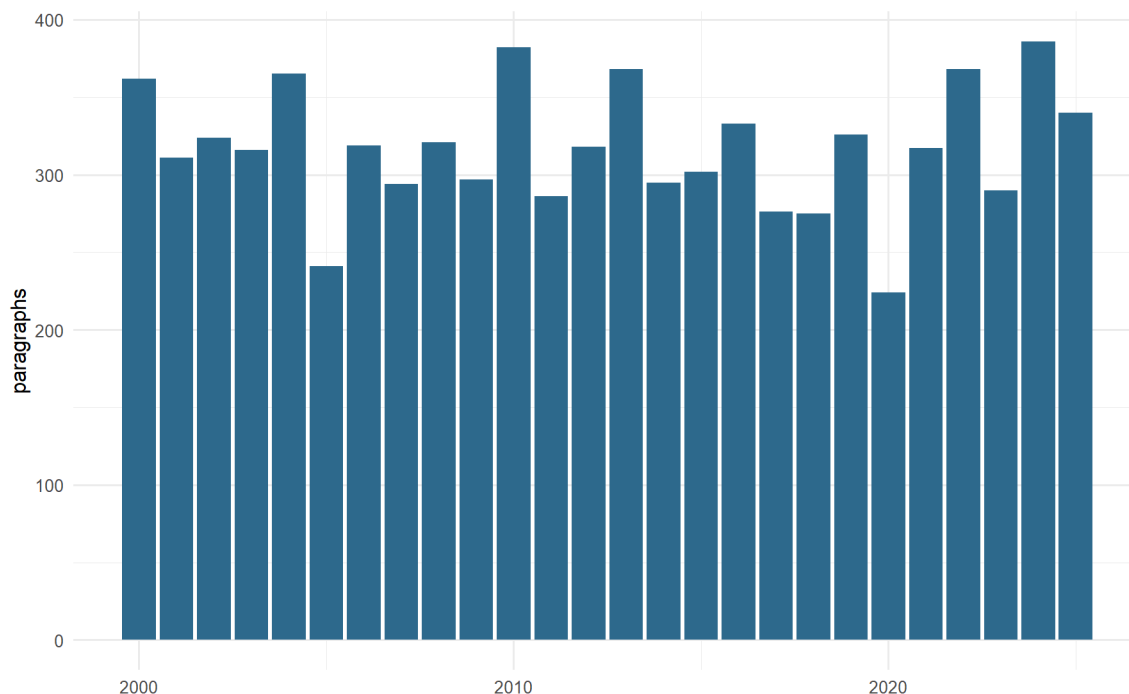


Figure 1: Paragraph volume by year: the denominator for every time-indexed estimate. Thin years mean the prevalence spline is interpolating and single-year claims are weak.

```
tokens <- paragraphs |>
  tidytext::unnest_tokens(word, text) |>
  dplyr::anti_join(tidytext::get_stopwords(), by = "word") |>
  dplyr::filter(str_detect(word, "[a-z]"))
```

Document-term matrix

Rare terms (under 5 paragraphs) are trimmed to stabilize estimation. `meta` is built from the surviving `dfm` rownames, so alignment with the model survives any dropped document. At work, check `nfeat(dfm_para)`: a vocabulary in the low thousands is comfortable at this scale, and a much larger one argues for a higher floor.

```
dfm_para <- tokens |>
  dplyr::count(para_uid, word) |>
  tidytext::cast_dfm(para_uid, word, n) |>
  quantda::dfm_trim(min_docfreq = tx$min_docfreq)
stm_in <- quantda::convert(dfm_para, to = "stm")
# as.data.frame, not a tibble: stm subsets metadata with base [ , ] semantics
```

```

# when building the prevalence model matrix, and a tibble returns a column
# rather than a vector there ("Error creating model matrix").
meta <- tibble(para_uid = rownames(dfm_para)) |>
  dplyr::left_join(paragraphs, by = "para_uid") |>
  as.data.frame()
stopifnot("every dfm row must match a paragraph" = !anyNA(meta$Year),
  "meta must align with the stm documents" =
    nrow(meta) == length(stm_in$documents))

# Spline basis as ORDINARY COLUMNS, so no function call ever appears inside a
# formula. stm builds its prevalence model matrix by evaluating the formula
# against the metadata, and a call there (s(), ns(), or anything namespaced)
# is the documented cause of "Error creating model matrix". prev_form then
# reads ~ yr1 + yr2 + ..., which is also what estimateEffect uses below.
bs <- splines::ns(meta$Year, df = tx$spline_df)
colnames(bs) <- str_c("yr", seq_len(ncol(bs)))
meta <- cbind(meta, as.data.frame(bs))
prev_rhs <- str_c(colnames(bs), collapse = " + ") # "yr1 + yr2 + ..."
prev_form <- as.formula(str_c("~ ", prev_rhs))

```

Choosing K

stm's own parallelism forks, which is unavailable on Windows, so the held-out grid is parallelized explicitly with furrr. Read the panels together: take the smallest K where held-out likelihood has flattened and residual dispersion has stopped falling, then break ties on the coherence-exclusivity frontier. If the frontier is still improving at the top of the grid, widen tx\$K_grid rather than extrapolating. Semantic coherence is $\sum_{i < j} \log\{(D(v_i, v_j) + 1)/D(v_i)\}$ over each topic's top words, with the denominator the higher-ranked word.

```

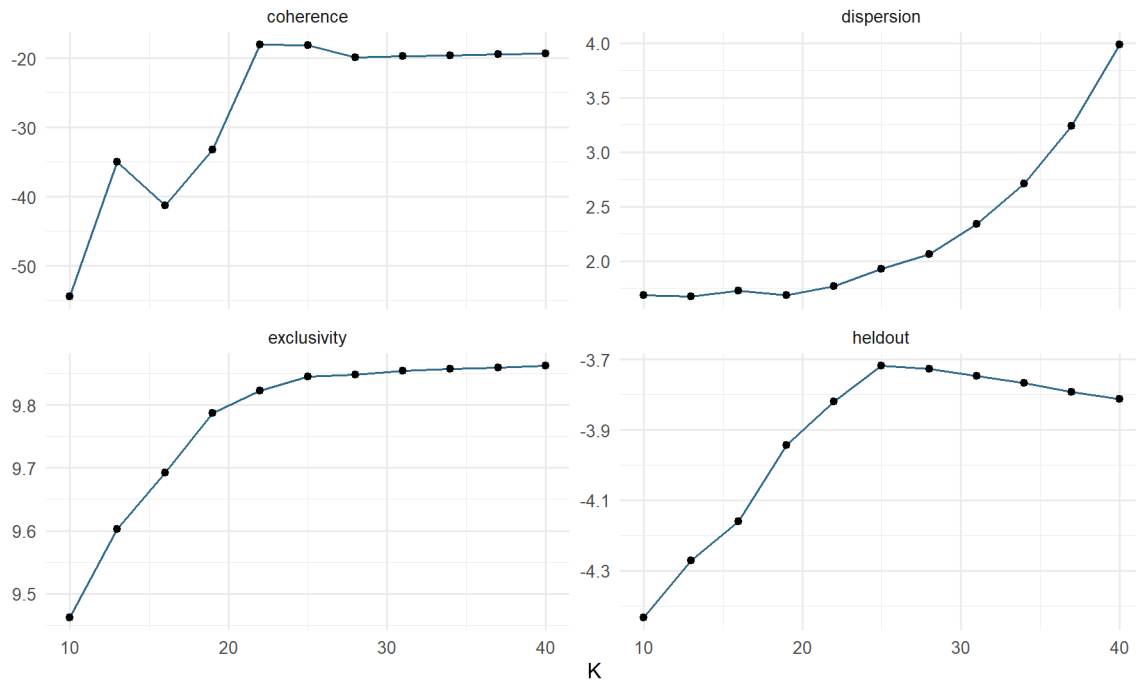
if (isTRUE(tx$run_searchK)) {
  future::plan(future::multisession, workers = tx$workers)
  ho <- stm::make.heldout(stm_in$documents, stm_in$vocab, seed = tx$seed)
  metrics <- cache_csv(p_out("searchK_metrics.csv"), {
    furrr::future_map(tx$K_grid, function(k) {
      fit <- stm::stm(ho$documents, ho$vocab, K = k, init.type = "Spectral",
        prevalence = prev_form, data = meta, verbose = FALSE)
      tibble(K = k,
        heldout = stm::eval.heldout(fit, ho$missing)$expected.heldout,
        dispersion = stm::checkResiduals(fit, ho$documents)$dispersion,
        coherence = mean(stm::semanticCoherence(fit, ho$documents)),
        exclusivity = mean(stm::exclusivity(fit)))
    }, .options = furrr::furrr_options(seed = tx$seed)) |> purrr::list_rbind()
  })
  future::plan(future::sequential)
  metrics |>
    tidyr::pivot_longer(-K) |>
    ggplot(aes(K, value)) + geom_line(color = viridis(1, begin = 0.35)) +

```

```

    geom_point() + facet_wrap(~ name, scales = "free_y") + labs(y = NULL)
  } else {
    if (is.null(tx$K))
      stop("tx$K is NULL and tx$run_searchK is FALSE. Re-run the config chunk, ",
           "or set tx$K before continuing.")
    glue::glue("searchK skipped (tx$run_searchK = FALSE); K fixed at {tx$K}")
  }
}

```



Structural topic model

Year enters the prevalence model as a spline (df 5), so topic attention is allowed to drift over the period instead of being averaged away; spectral initialization is deterministic given the data, removing initialization luck from a fit everything downstream depends on.

```

stm_fit <- cache_rds(p_out("stm_fit.rds"),
  stm::stm(stm_in$documents, stm_in$vocab, K = tx$K, init.type = "Spectral",
    prevalence = prev_form, data = meta, verbose = FALSE))

theta <- stm_fit$theta
colnames(theta) <- str_c("topic_", seq_len(tx$K))
paragraph_theta <- meta |> dplyr::bind_cols(tibble::as_tibble(theta))

```

Modal topic assignment and its diagnostics

Each paragraph is assigned its **modal** topic, so topics partition the corpus (mutually exclusive, collectively exhaustive) and can serve as sampling strata in Script 2. This is the same move as modal class assignment in latent class analysis, and it inherits the same caveat: a modal assignment is an estimate, not an observation, and treating it as observed pushes classification error downstream. Two LCA diagnostics report how much error to expect. **Entropy** R^2 summarizes overall assignment sharpness on a 0-1 scale; **AvePP**, the average posterior probability among a topic's own assigned paragraphs, does it per topic, with about 0.7 the conventional floor. Unlike LCA, STM supplies no classification error matrix over modal assignment, so no BCH-style correction is available here; the honest substitutes are the `min_theta` scope floor in Script 2 and its sensitivity analysis.

```
paragraph_theta <- paragraph_theta |>
  dplyr::mutate(topic      = max.col(theta, ties.method = "first"),
               theta_max = theta[cbind(dplyr::row_number(), topic)])

ent <- -sum(theta * log(theta + 1e-12))
entropy_r2 <- 1 - ent / (nrow(theta) * log(tx$K))

topic_summary <- tibble(
  topic = seq_len(tx$K),
  prevalence = colMeans(theta),
  frex = apply(stm::labelTopics(stm_fit, n = 8)$frex, 1, str_c,
               collapse = ", "),
  prob = apply(stm::labelTopics(stm_fit, n = 8)$prob, 1, str_c,
               collapse = ", ")) |>
  dplyr::left_join(
    paragraph_theta |>
      dplyr::group_by(topic) |>
      dplyr::summarise(n paras = dplyr::n(), avepp = mean(theta_max)),
    by = "topic")

readr::write_csv(topic_summary, p_out("topic_summary.csv"))
readr::write_csv(paragraph_theta, p_out("paragraph_theta.csv"))

glue::glue("Entropy R2 = {round(entropy_r2, 3)}; topics with AvePP < 0.7: ",
           "{sum(topic_summary$avepp < 0.7)} of {tx$K}")
```

```
Entropy R2 = 0.905; topics with AvePP < 0.7: 0 of 25
```

```
topic_summary |>
  dplyr::mutate(dplyr::across(c(prevalence, avepp), ~ round(.x, 3))) |>
  dplyr::arrange(dplyr::desc(prevalence)) |>
  knitr::kable(caption = "Topics by prevalence with FREX and highest-probability")
```

terms, assigned paragraph counts, and AvePP. Script 2 labels these; the terms here are the evidence a human uses to check those labels.")

topic	prevalence	freq	prob	n_paras	avepp
24	0.046	t25term1t25term10, t25term2t25term2, t25term9t25term9, t25term5t25term5, t25term6t25term6, t25term7t25term7, t25term8t25term8, t25term1t25term1		381	0.963
13	0.045	t22term5t22term5, t22term9t22term9, t22term1t22term10, t22term1t22term1, t22term8t22term8, t22term6t22term6, t22term4t22term4, t22term1t22term12		370	0.962
22	0.045	t2term2, t2term2, t2term1t2term11, t2term12t2term12, t2term3, t2term3, t2term7, t2term7, t2term10t2term10, t2term1, t2term1, t2term4 t2term4		374	0.962
23	0.045	t19term1t19term1, t19term9t19term9, t19term6t19term6, t19term1t19term11, t19term1t19term12, t19term1t19term10, t19term3t19term3, t19term4t19term4		367	0.962
25	0.044	t23term1t23term5, t23term1t23term10, t23term1t23term1,		375	0.897

topic	prevalence	frex	prob	n_paras	avepp
			t23term8t23term6, t23term4t23term12, t23term5t23term11, t23term7t23term4, t23term1t23term9		
15	0.043		t4term7, t4term7, t4term11t4term11, t4term2, t4term2, t4term5, t4term5, t4term8, t4term8, t4term1, t4term1, t4term12t4term12, t4term10t4term10	352	0.962
16	0.043		t24term1t24term1, t24term9t24term9, t24term1t24term11, t24term6t24term6, t24term4t24term4, t24term8t24term8, t24term5t24term5, t24term1t24term10	351	0.962
20	0.043		t1term5, t1term5, t1term8, t1term8, t1term3, t1term3, t1term2, t1term2, t1term9, t1term9, t1term7, t1term7, t1term6, t1term6, t1term1 t1term1	355	0.962
2	0.041		t6term1, t6term1, t6term5, t6term5, t6term8, t6term8, t6term11t6term11, t6term10t6term10, t6term6, t6term6, t6term9, t6term9, t6term7 t6term7	335	0.961

topic	prevalence	freq	prob	n paras	avepp
5	0.041	t21term1	t21term1, t21term11, t21term2, t21term12, t21term5, t21term4, t21term9, t21term8	339	0.961
7	0.041	t20term9	t20term9, t20term12, t20term3, t20term10, t20term11, t20term4, t20term5, t20term6	338	0.961
21	0.041	t7term11	t7term11, t7term6, t7term6, t7term7, t7term7, t7term2, t7term2, t7term10, t7term10, t7term5, t7term5, t7term4, t7term4, t7term9 t7term9	335	0.961
11	0.040	t3term6	t3term6, t3term6, t3term12, t3term12, t3term2, t3term2, t3term3, t3term3, t3term8, t3term8, t3term9, t3term9, t3term10, t3term10, t3term4 t3term4	326	0.961
3	0.039	t5term2	t5term2, t5term2, t5term1, t5term1, t5term12, t5term12, t5term7, t5term7, t5term11, t5term11, t5term5, t5term5,	321	0.961

topic	prevalence	freq	prob	n paras	avepp
		t5term10, t5term10, t5term9 t5term9			
6	0.039	t16term5, t16term5, t16term7, t16term7, t16term4, t16term4, t16term1, t16term1, t16term9, t16term9, t16term12, t16term12, t16term3, t16term3, t16term8, t16term8		319	0.960
10	0.038	t10term1, t10term1, t10term12, t10term12, t10term11, t10term11, t10term5, t10term5, t10term8, t10term8, t10term2, t10term2, t10term4, t10term4, t10term9, t10term9		312	0.960
18	0.038	t17term1, t17term1, t17term11, t17term11, t17term9, t17term9, t17term6, t17term6, t17term7, t17term7, t17term8, t17term8, t17term10, t17term10, t17term3, t17term3		315	0.960
4	0.037	t12term4, t12term4, t12term7, t12term7, t12term12, t12term12, t12term11, t12term11, t12term2, t12term2, t12term6, t12term6, t12term5, t12term5, t12term8, t12term8		301	0.960
12	0.037	t9term8, t9term8, t9term2, t9term2, t9term9, t9term9, t9term7, t9term7,		300	0.960

topic	prevalence	freq	prob	n paras	avepp
		t9term6, t9term6, t9term3, t9term3, t9term12, t9term12, t9term5 t9term5			
14	0.037	t11term2, t11term2, t11term11, t11term11, t11term10, t11term10, t11term1, t11term1, t11term12, t11term12, t11term9, t11term9, t11term3, t11term3, t11term5, t11term5		303	0.960
9	0.036	t18term2, t18term2, t18term7, t18term7, t18term8, t18term8, t18term6, t18term6, t18term10, t18term10, t18term3, t18term3, t18term5, t18term5, t18term9, t18term9		299	0.960
17	0.036	t14term11, t14term11, t14term1, t14term1, t14term8, t14term8, t14term12, t14term12, t14term4, t14term4, t14term2, t14term2, t14term3, t14term3, t14term5, t14term5		296	0.960
19	0.036	t13term1, t13term1, t13term7, t13term7, t13term3, t13term3, t13term12, t13term12, t13term5, t13term5, t13term9, t13term9, t13term11, t13term11, t13term10, t13term10		297	0.960
1	0.035	t15term11, t15term11, t15term10, t15term10,		289	0.960

topic	prevalence	frex	prob	n_paras	avepp
		t15term9, t15term5, t15term8, t15term3, t15term4, t15term1			
8	0.035	t8term10, t8term5, t8term9, t8term1, t8term12, t8term2, t8term3, t8term11		286	0.960

Table 2: Topics by prevalence with FREX and highest-probability terms, assigned paragraph counts, and AvePP. Script 2 labels these; the terms here are the evidence a human uses to check those labels.

Representative paragraphs

```
top4 <- topic_summary |> dplyr::slice_max(prevalence, n = 4) |>
  dplyr::pull(topic)
th <- stm::findThoughts(stm_fit, texts = meta$para_uid, n = 3, topics = top4)
tibble(topic = rep(top4, each = 3), para_uid = unlist(th$docs)) |>
  dplyr::left_join(paragraphs |> dplyr::select(para_uid, text),
    by = "para_uid") |>
  dplyr::mutate(text = str_trunc(text, 150)) |>
  knitr::kable(caption = "Three highest-theta paragraphs for the four most
prevalent topics.")
```

topic	para_uid	text
24	A00206-03	t25term6 t25term2 t25term5 t25term4 t25term12 t25term6 t25term3 t25term2 t25term9 t25term7 t25term7 t25term5 t25term3 t25term3 common2 common29 com...
24	A00346-05	t25term5 t25term1 t25term11 t25term3 t25term2 t25term5 t25term8 t25term12 t25term2 t25term4 t25term4 t25term9 t25term10 t25term11 common6 common38 ...

topic	para_uid	text
24	A00487-05	t25term8 t25term4 t25term9 t25term3 t25term10 t25term10 t25term3 t25term5 t25term10 t25term4 t25term12 t25term9 t25term8 t25term7 common7 common26 ...
22	A00012-03	t2term7 t2term7 t2term1 t2term8 t2term9 t2term8 t2term6 t2term1 t2term5 t2term5 t2term6 t2term1 t2term11 t2term10 common31 com- mon23 common35 common...
22	A01023-09	t2term4 t2term3 t2term9 t2term6 t2term9 t2term4 t2term11 t2term6 t2term10 t2term2 t2term7 t2term12 t2term4 t2term12 common27 common14 common35 comm...
22	A00052-01	t2term6 t2term8 t2term12 t2term6 t2term6 t2term2 t2term1 t2term6 t2term7 t2term12 t2term4 t2term9 t2term3 t2term5 common27 com- mon25 common23 common...
13	A00983-05	t22term1 t22term3 t22term2 t22term4 t22term12 t22term5 t22term7 t22term4 t22term9 t22term5 t22term7 t22term9 t22term8 t22term6 common18 common8 com...
13	A00328-03	t22term1 t22term11 t22term9 t22term5 t22term10 t22term3 t22term9 t22term2 t22term8 t22term6 t22term9 t22term10 t22term5 t22term3 common6 common18 c...
13	A00723-08	t22term1 t22term9 t22term5 t22term3 t22term3 t22term11 t22term10 t22term9 t22term12 t22term6 t22term12 t22term11 t22term11 t22term3 common12 common...
23	A00113-01	t19term3 t19term10 t19term3 t19term7 t19term10 t19term10 t19term6 t19term10 t19term10 t19term2 t19term11 t19term2 t19term8 t19term7 common3 common1...
23	A00277-05	t19term8 t19term11 t19term10 t19term3 t19term9 t19term7 t19term5 t19term5 t19term10 t19term8 t19term2 t19term4 t19term6 t19term12 common12 common18...
23	A00487-09	t19term1 t19term6 t19term3 t19term11 t19term6 t19term8 t19term1 t19term7 t19term5 t19term4 t19term3 t19term3 t19term11 t19term5 common10 common10 c...

Table 3: Three highest-theta paragraphs for the four most prevalent topics.

Prevalence over time

```
stm_year_effects <- cache_rds(p_out("stm_year_effects.rds"),
  stm::estimateEffect(
    as.formula(str_c("1:", tx$K, " ~ ", prev_rhs)),
    stm_fit, metadata = meta, uncertainty = "Global"))
```

Expected topic prevalence is predicted across the observed year range from the fitted effect object, with 95 percent intervals. Two details make the prediction correct rather than merely plausible. The spline basis is rebuilt at the prediction grid with `predict(bs, ...)`, which reuses the **original knots**; calling `ns()` afresh on the grid would place knots differently and mis-predict silently. And the intervals come from the simulation draws `estimateEffect` already stores under `uncertainty = "Global"`, so they carry the topic-model uncertainty rather than regression standard errors alone.

```
# Prediction grid: every observed year, with the basis evaluated at the
# ORIGINAL knots so the fitted coefficients apply.
grid <- tibble(Year = sort(unique(meta$Year)))
Xg    <- cbind(1, predict(bs, newx = grid$Year))

# estimateEffect stores per-topic simulation draws of the coefficient vector
# in $parameters[[k]][[j]]$est. Predicting each draw and taking quantiles
# propagates both the regression and the topic-model uncertainty.
draws_for_topic <- function(k) {
  purrr::map(stm_year_effects$parameters[[k]], function(sim)
    as.numeric(Xg %*% sim$est)) |>
  purrr::reduce(rbind) |> matrix(ncol = nrow(grid))
}
topic_trends <- purrr::map(seq_along(stm_year_effects$topics), function(i) {
  d <- draws_for_topic(i)
  grid |> dplyr::mutate(
    topic = stm_year_effects$topics[i],
    fit    = apply(d, 2, mean),
    lo     = apply(d, 2, quantile, 0.025),
    hi     = apply(d, 2, quantile, 0.975))
}) |> purrr::list_rbind()
readr::write_csv(topic_trends, p_out("topic_trends.csv"))
glue::glue("Predicted prevalence for {dplyr::n_distinct(topic_trends$topic)} \
\
      topics across {nrow(grid)} years written to topic_trends.csv")
```

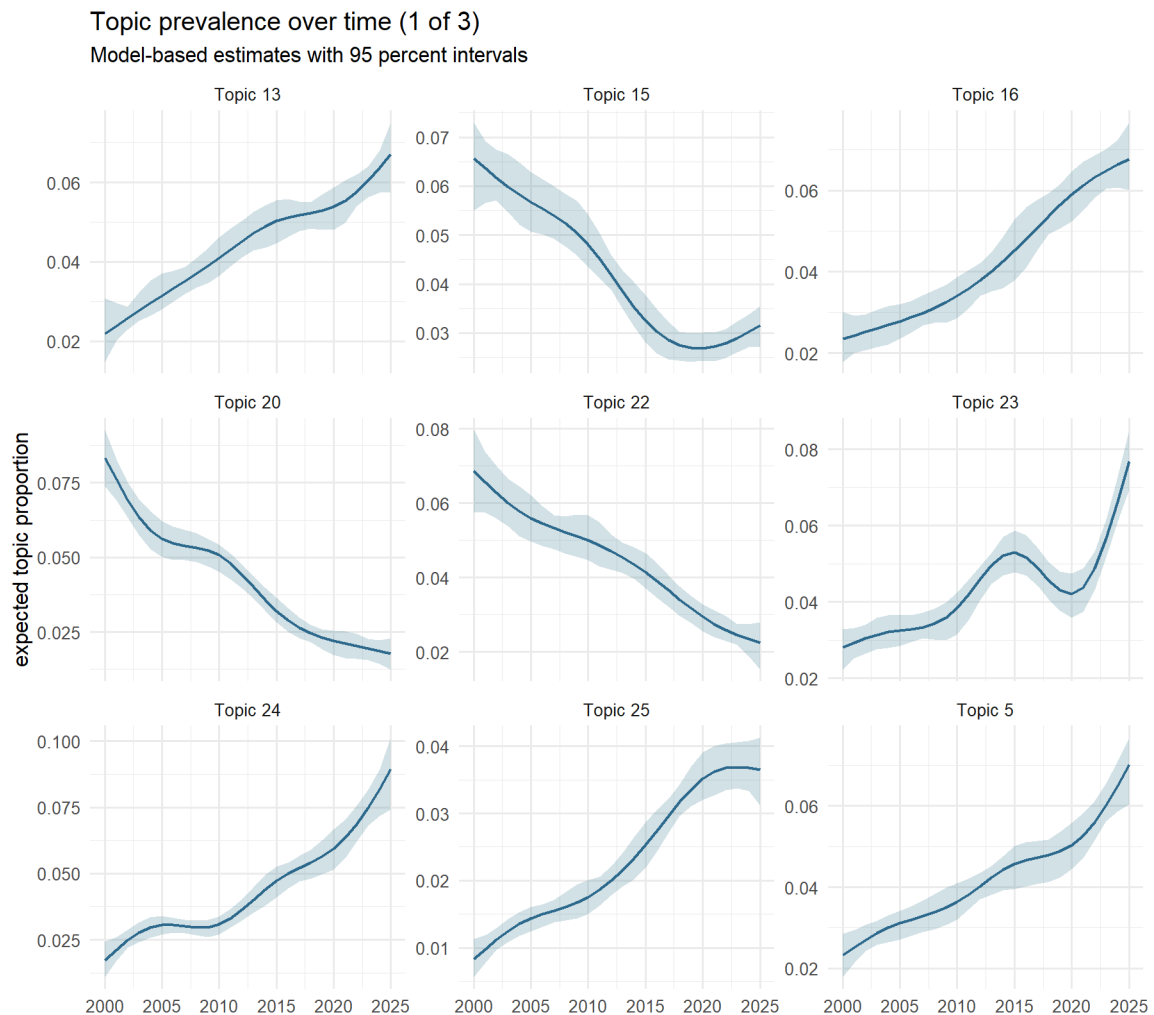
Predicted prevalence for 25 topics across 26 years written to topic_trends.csv

```
# All topics, paged so panels stay legible: 25 facets on one page renders each
# about 1.5 inches wide and the ribbons become unreadable.
trend_panel <- function(d, page, n_pages) {
  ggplot(d, aes(Year, fit)) +
    geom_ribbon(aes(ymin = lo, ymax = hi), alpha = 0.2,
              fill = viridis(1, begin = 0.35)) +
    geom_line(linewidth = 0.7, color = viridis(1, begin = 0.35)) +
    facet_wrap(~ topic_label, scales = "free_y", ncol = 3) +
    labs(x = NULL, y = "expected topic proportion",
         title = glue::glue("Topic prevalence over time ({page} of {n_pages})"),
```

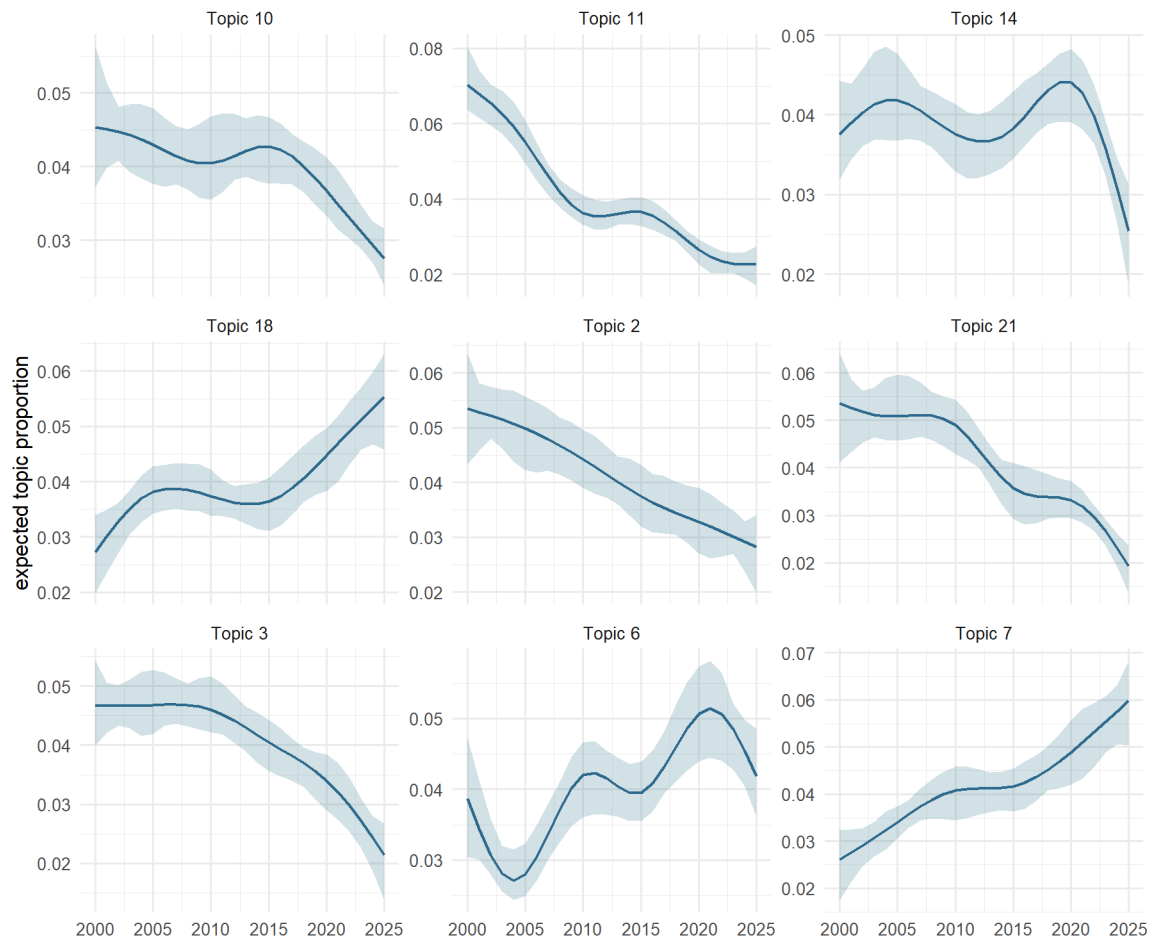
```

      subtitle = "Model-based estimates with 95 percent intervals")
}
trend_pages <- topic_trends |>
  dplyr::left_join(topic_summary |> dplyr::select(topic, prevalence),
    by = "topic") |>
  dplyr::mutate(
    topic_label = str_c("Topic ", topic),
    page = (dplyr::dense_rank(dplyr::desc(prevalence)) - 1L) %/%
      tx$facets_per_page + 1L) |>
  dplyr::group_split(page)
purrr::iwalk(trend_pages, function(d, i)
  print(trend_panel(d, i, length(trend_pages))))

```



Topic prevalence over time (2 of 3)
Model-based estimates with 95 percent intervals



Topic prevalence over time (3 of 3)
Model-based estimates with 95 percent intervals



```
glue::glue("outputs/ written: paragraphs.csv, paragraph_theta.csv (ids, Year, \
\
n_tokens, text, topic_1..K, topic, theta_max), topic_summary.csv, \
stm_fit.rds, stm_year_effects.rds. Script 2 reads these.")
```

```
outputs/ written: paragraphs.csv, paragraph_theta.csv (ids, Year, n_tokens,
text, topic_1..K, topic, theta_max), topic_summary.csv, stm_fit.rds,
stm_year_effects.rds. Script 2 reads these.
```